

UNIT - V

5

Cloud Platforms for IoT

Syllabus

Software Defined Networking, Introduction to Cloud Storage Models, Communication API. **WAMP** : Auto Bahn for IoT, Xively Cloud for IoT. **Python Web Application Framework** : Django Architecture and application development with Django, Amazon Web Services for IoT, Sky Net IoT Messaging Platform, RESTful Web Service, GRPC, SOAP.

Contents

- 5.1 Software Defined Networking
- 5.2 Introduction to Cloud **Dec.-19**.....Marks 3
- 5.3 WAMP : Auto Bahn for IoT **May-18, 19, Dec.-18, 19**.....Marks 7
- 5.4 Xively Cloud for IoT **May-18, 19, Dec.-18, 19**.....Marks 6
- 5.5 Python Web Application Framework : Django
..... **Dec.-18, May-19**.....Marks 6
- 5.6 Amazon Web Services for IoT **May-18, 19, Dec.-18, 19**.....Marks 7
- 5.7 Sky Net IoT Messaging Platform
- 5.8 RESTful Web Service **March - 20**.....Marks 5
- 5.9 gRPC
- 5.10 Simple Object Access Protocol
- 5.11 Multiple Choice Questions

5.1 Software Defined Networking

- The physical separation of the network control plane from the data plane and where a control plane controls several devices.
- The centralized management facilitates optimization and configuration of a network in an efficient and automated manner and provides interoperability among heterogeneous IoT network.
- SDN is a framework to allow network administrators to automatically and dynamically manage and control a large number of network devices, services, topology, traffic paths, and packet handling policies using high-level languages and APIs.
- Router can be partitioned into control and data plane. Network intelligence and state are logically centralized. The underlying network infrastructure is abstracted from the applications.
- Execute or run Control plane software on general purpose hardware. Architecture to control not just a networking device but an entire network.
- SDN is basically layer architecture consists of three layers :
 - 1) Device layer or data plane
 - 2) Control plane and
 - 3) Application layer
- Software defined network is a network architecture that separates the control plane from the data plane and centralizes the network controller
- Software Defined Networking (SDN) allows network engineers to support a switching fabric across multi-vendor hardware and application-specific integrated circuits.

Limitation of Conventional Network Architecture

- In the traditional approach to networking, most network functionality is implemented in a dedicated appliance; i.e., switch, router, application delivery controller.
- In addition, within the dedicated appliance, most of the functionality is implemented in dedicated hardware such as an ASIC (Application Specific Integrated Circuit).
- Some of the key characteristics of this approach to developing network appliances are :
 1. The ASICs that provide the network functionality evolve slowly;
 2. The evolution of ASIC functionality is under the control of the provider of the appliance;
 3. The appliances are proprietary;
 4. Each appliance is configured individually;
 5. Tasks such as provisioning change management and de-provisioning are very time consuming and error prone

❑ Software Defined Network

- Software defined network is a network architecture that separates the control plane from the data plane and centralizes the network controller.
- Software Defined Networking (SDN) allows network engineers to support a switching fabric across multi-vendor hardware and application-specific integrated circuits.
- A network administrator can shape traffic from a centralised control console without having to touch individual switches.
- SDN is also useful for maintaining service level in a cloud environment, by providing an API-enabled approach to manage the network and move workloads in a consistent way.
- The physical separation of the network control plane from the data plane (forwarding plane), and where a control plane controls several devices.
- The centralized management facilitates optimization and configuration of a network in an efficient and automated manner and provides interoperability among heterogeneous IoT network.
- SDN is a framework to allow network administrators to automatically and dynamically manage and control a large number of network devices, services, topology, traffic paths, and packet handling policies using high-level languages and APIs.
- Router can be partitioned into control and data plane. Network intelligence and state are logically centralized. The underlying network infrastructure is abstracted from the applications.
- Execute or run control plane software on general purpose hardware. Architecture to control not just a networking device but an entire network.
- Fig. 5.1.1 show an abstract model of a traditional switch and a switch with separated planes.

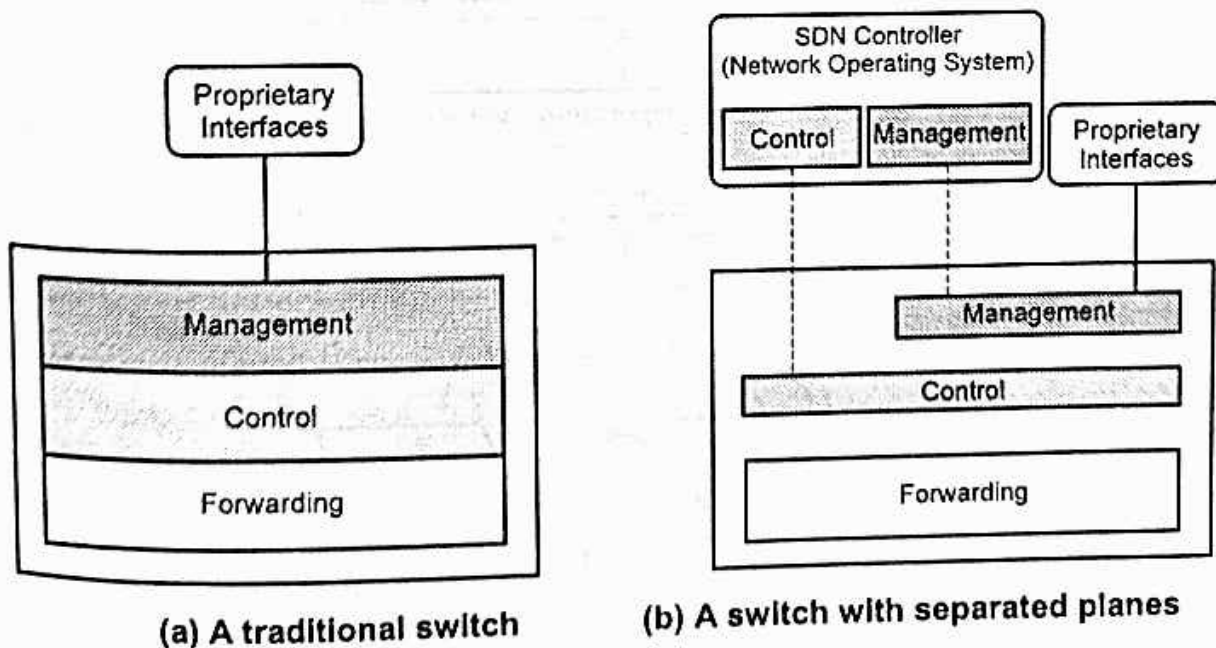


Fig. 5.1.1

5.1.1 SDN Architecture

- SDN is basically layer architecture consists of three layers :
 - 1) Device layer or data plane
 - 2) Control plane and
 - 3) Application layer
- The SDN architecture is commonly defined by four principles :
 - (i) The separation of the control and data planes.
 - (ii) Flow-based forwarding in the data plane.
 - (iii) Logically centralized SDN controller that instructs multiple forwarding planes.
 - (iv) Network programmability via the SDN controller.
- In the SDN architecture the control plane is physically decoupled from the data forwarding plane. The data plane's forwarding behavior is determined by a set of flow tables, and the forwarding decisions are flow-based.
- A logically centralized SDN controller implements the control plane logic and remotely controls several data planes. The SDN controller builds a global view of the network, and offers a control logic that programs an abstract model of this view.
- Fig. 5.1.2 shows SDN Architecture.

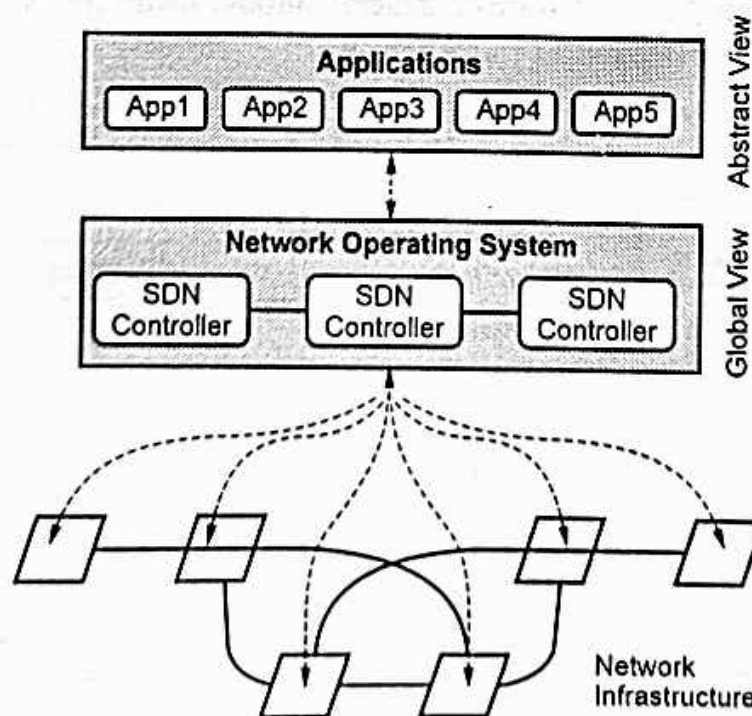


Fig. 5.1.2 : SDN architecture

- SDN is a layered architecture, consisting of three basic layers; application / services layer, a controller layer, and data plane layer called forwarding layer consisting of forwarding devices.
- These SDN layers communicate with each other via open APIs called Northbound Interface (NI) API and Southbound Interface (SI) API.
- **Application layer (AP)** : The application plane also called management plane consist of applications that leverage the functions offered by the NI to implement network control and operation logic.
- The Network Operating System (NOS) facilitate application developers to coordinate through these NI APIs. Typically, an NI APIs abstracts the low-level instruction sets and implementation of forwarding devices.
- Control plane is the decoupled entity from the distributed forwarding devices and logically centralized on a server.
- Southbound interfaces provide a communication protocol between CP and forwarding device though the SI instruction set.
- Forwarding devices (routers, switches, gateways etc.) are interconnected through a physical medium such as wireless radio channels or wired cables.
- In SDN architecture, the SDN controller is responsible for building a global view of the network and providing mechanisms for network management and control.
- The SDN architecture is endorsing a logically centralized model. However, it does not contend for a physically centralized implementation.
- The SDN architecture is :
 1. **Directly programmable** : Network control is directly programmable because it is decoupled from forwarding functions.
 2. **Agile** : Abstracting control from forwarding lets administrators dynamically adjust network-wide traffic flow to meet changing needs.
 3. **Centrally managed** : Network intelligence is (logically) centralized in software-based SDN controllers that maintain a global view of the network, which appears to applications and policy engines as a single, logical switch.
 4. **Programmatically configured** : SDN lets network managers configure, manage, secure, and optimize network resources very quickly via dynamic, automated SDN programs, which they can write themselves because the programs do not depend on proprietary software.

- 5. Open standards-based and vendor-neutral :** When implemented through open standards, SDN simplifies network design and operation because instructions are provided by SDN controllers instead of multiple, vendor-specific devices and protocols.

5.1.2 SDN Layers

- Fig. 5.1.3 shows SDN layers.

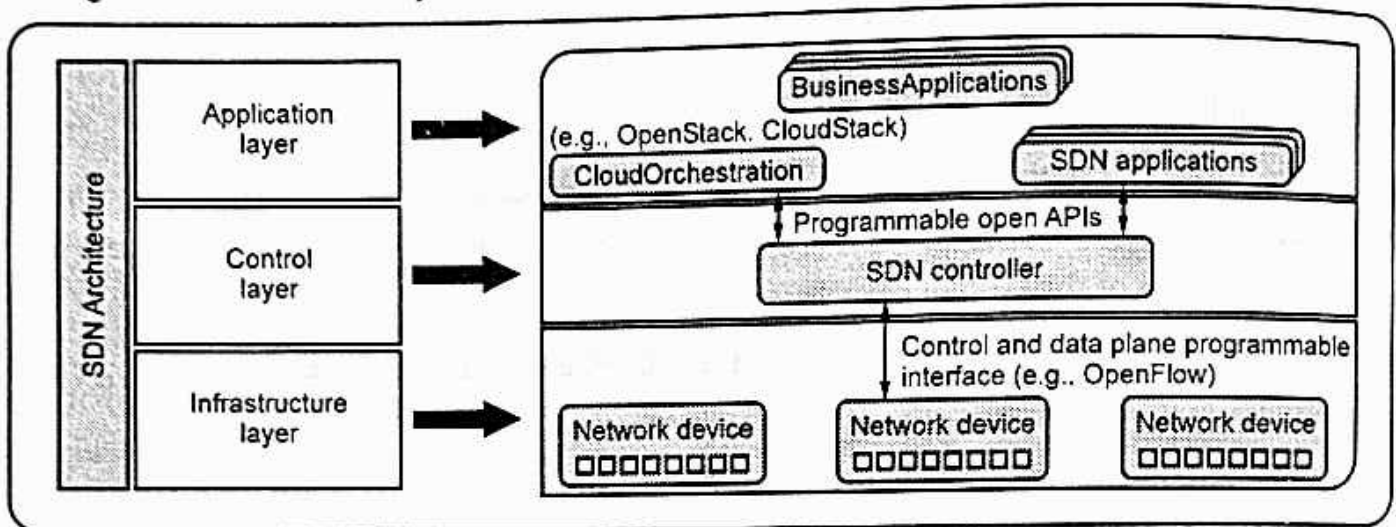


Fig. 5.1.3 : SDN layers

- Infrastructure layer** acts as the foundation for an SDN architecture. The infrastructure consists of both physical and virtual network devices such as switches and routers. These devices implement the OpenFlow protocol as a standards-based method of implementing traffic forwarding rules.
- Control layer** consists of a centralized control plane for the entire network. The control plane is decoupled from the underlying infrastructure to provide a single centralized view of the entire network. The control layer utilizes OpenFlow to communicate with the infrastructure layer.
- Application layer** consists of network services and business applications that interact with the control layer. These applications leverage open interfaces to communicate with the control layer and the network state.

5.1.3 OpenFlow

- OpenFlow is a Layer 2 communications protocol that gives access to the forwarding plane of a network switch or router over the network. Fig. 5.1.4 shows components of an OpenFlow Switch.

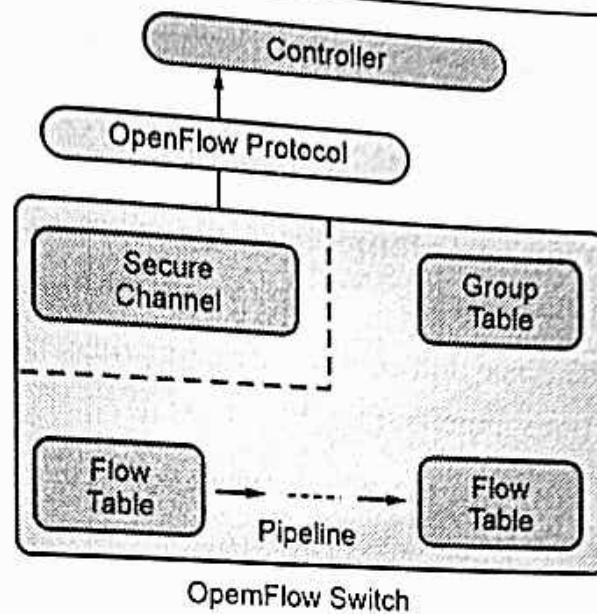


Fig. 5.1.4 : Components of an OpenFlow Switch

- The number of flows a switch can handle is limited by the size of its flow table. Secure Channel (SC) is the interface that connects each OpenFlow switch to controller. A controller configures and manages the switch via this interface.
- SC establishes and terminates the connection between OpenFlow Switch and the controller using the procedures.
- Each OpenFlow switch has a chain of flow tables, and each table stores a collection of flow entries. A flow is defined as the set of packets that match the given properties, e.g., a particular pair of source and destination MAC addresses. A flow entry defines the forwarding/routing rules. It consists of a bit pattern that indicates the flow properties, a list of actions, and a set of counters.

5.1.4 Network Foundation Virtualization

- Network Function Virtualization (NFV) is an effort to decouple the network function logic from the underlying hardware, such that the network devices are consolidated onto regular servers, switches, and storage.
- NFV decreases costs by consolidating network services onto commodity hardware, taking advantage of the economies of scale.
- NFV is way to reduce cost and accelerate service deployment for network operators by decoupling functions like a firewall or encryption from dedicated hardware and moving them to virtual servers.
- NFV and SDN are complementary. One does not depend upon the other. You can do SDN only, NFV only, or SDN and NFV.

- NFV allows use of a single physical platform for different applications, users and tenants. It enables new ways to implement resilience, service assurance, test & diagnostics and security surveillance.
- Facilitates innovation towards new network functions and services that are only practical in a pure software network environment.
- It is applicable to any data plane and control plane functions. Automation of management and configuration of functions important for NFV to scale.
- NFV aims to ultimately transform the way network operators architect and operate their networks - though change will be incremental.
- Fig. 5.1.5 shows NFV architecture.

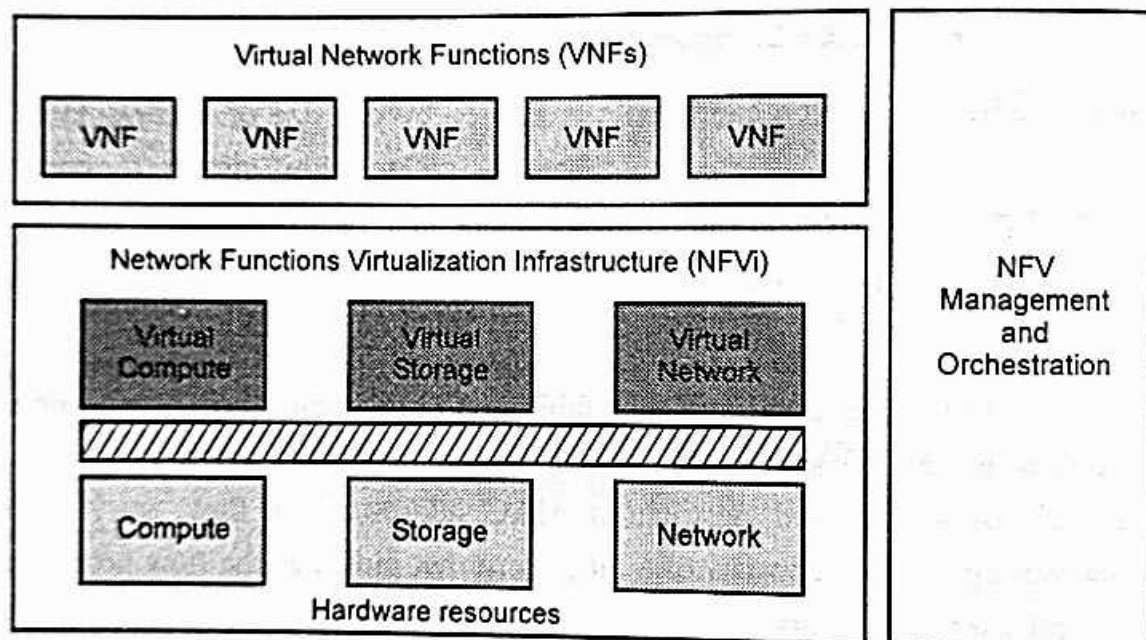


Fig. 5.1.5 : NFV architecture

- Elements of NFV architecture are as follows :
 1. **Virtualized Network Function** : It implements network function in software and it runs over NFV infrastructure.
 2. **NFV Infrastructure** : It contains network, storage and compute resources that are virtualized. It is the combination of both hardware and software resources which build up the environment in which VNFs are deployed, managed and executed.
 3. The Network Functions Virtualisation Management and Orchestration (NFV-MANO) architectural framework has the role to manage the NFVI and orchestrate the allocation of resources needed by the NSs and VNFs.

- Instead of installing expensive proprietary hardware, service providers can purchase inexpensive switches, storage and servers to run virtual machines that perform network functions. This collapses multiple functions into a single physical server, reducing costs and minimizing truck rolls.
- If a customer wants to add a new network function, the service provider can simply spin up a new virtual machine to perform that function.
- For example, instead of deploying a new hardware appliance across the network to enable network encryption, encryption software can be deployed on a standardized server or switch already in the network.
- This virtualization of network functions reduces dependency on dedicated hardware appliances for network operators, and allows for improved scalability and customization across the entire network. Different from a virtualized network, NFV seeks to offload network functions only, rather than the entire network.

❑ Benefits :

1. Reduced equipment costs and reduced power consumption
2. NFV allows to abstract underlying hardware, and enables elasticity, scalability and automation.
3. Increased speed of deployment by minimizing the typical network operator cycle of innovation
4. Services can be rapidly scaled up/down as required
5. Improved operational efficiency

5.2 Introduction to Cloud

➡ SPPU : Dec.-19

- Cloud computing refer to a variety of services available over the Internet that deliver compute functionality on the service provider's infrastructure. Its environment (infrastructure) may actually be hosted on either a grid or utility computing environment, but that doesn't matter to a service user.
- Using the Internet for communication and transport provides hardware, software and networking services to clients. These platforms hide the complexity and details of the underlying infrastructure from users and applications by providing very simple graphical interface or API.
- Cloud storage delivers virtualized storage on demand, over a network based on a request for a given QoS. There is no need to purchase storage or in some cases even provision it before storing data. You only pay for the amount of storage your data is actually consuming.

- Cloud storage is a model of networked online storage where data is stored in virtualized pools of storage which are generally hosted by third parties. Hosting companies operate large data centers and people who require their data to be hosted buy or lease storage capacity from them.
- Cloud storage services may be accessed through a web service API, a cloud storage gateway or through a web-based user interface.
- Several large Web companies (such as Amazon and Google) are now exploiting the fact that they have data storage capacity that can be hired out to others. This approach, known as cloud storage allows data stored remotely to be temporarily cached on desktop computers, mobile phones or other Internet-linked devices. Amazon's Elastic Compute Cloud (EC2) and Simple Storage Solution (S3) are well known examples.
- Cloud storage advantages
 1. Organizations need only pay for the storage they actually use as it is also possible for companies by utilizing actual virtual storage features like thin provisioning.
 2. Companies do not need to install physical storage devices in their own data centre or offices.
 3. Storage maintenance tasks, such as backup, data replication and purchasing additional storage devices are offloaded to the responsibility of a service provider, allowing organizations to focus on their core business.
 4. Cloud storage provides users with immediate access to a broad range of resources and applications hosted in the infrastructure of another organization via a web service interface.
 5. Cloud storage can be used for copying virtual machine images from the cloud to on-premise locations or to import a virtual machine image from an on-premise location to the cloud image library.
- A storage model describes the layout of a data structure in physical storage; a data model captures the most important logical aspects of a data structure in a database. The physical storage can be a local disk, a removable media, or storage accessible via a network.
- Popular Models are Amazon Web Service (AWS) and Xively Cloud (PAAS)



5.2.1 Communication API

- Cloud Models are relied on Communication API.
- Communication API support data transfer, control information transfer from application to cloud, one service to another.

- It also exist in the form of Communication Protocols. It supports RPC, PUBSUB and WAMP.
- Django web framework is used to implement Communication API

Review Question

1. Write a short note on cloud storage models and communication APIs.

SPPU : Dec.-19, Marks 3

5.3 WAMP : Auto Bahn for IoT

SPPU : May-18, 19, Dec.-18, 19

- WAMP stands for Web Application Messaging Protocol. It is sub-protocol of web-socket and provides publish-subscribe and RPC messaging pattern. WAMP is a routed protocol, with all components connecting to a WAMP Router, where the WAMP Router performs message routing between the components.
- It support distributed application architecture. Application components are distributed and run on mobile nodes and WAMP provides communication in between them.
- WAMP does not make any extra standard, but it bases on established Web standards : WebSocket, JSON and URI. The WebSocket standard is the default transport channel where WAMP is binding. With the WebSocket Standard, WAMP is assumed to be given a reliable, ordered and full-duplex message channel.
- The PubSub messaging pattern defines three roles : Subscribers and Publishers, which communicate via a Broker. The routed RPC messaging pattern also defines three roles : Callers and Calleees, which communicate via a Dealer.
- Key components of WAMP :
 1. Transport : It is communication channel which connects two peers. WebSocket is default transport for WAMP. WAMP also uses other transport protocol which is reliable and bi-directional communication.
 2. Session : Conversation between two is called as session.
 3. Client : Clients are peers that can have one or more roles. In publish-subscribe model, client can have following roles :
 - Message producers are called publishers, whereas message consumers are called subscribers. Messages published to a topic using the publish-and-subscribe model can be received by multiple subscribers. Every subscriber receives a copy of each message, message is broadcasted. Fig. 5.3.1 shows publish-subscribe model.

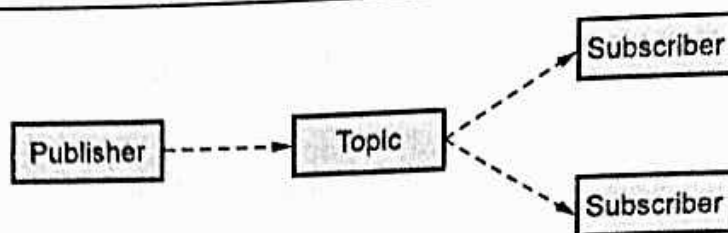


Fig. 5.3.1 : Publish-subscribe model

- Role of client in RPC model is as follows :
 1. The caller program, which represents the client instance in the client/server model sends a call message to the server process, and waits for a reply message.
 2. Callee : Callee executes the producer to which the calls are issued by the caller and returns the result back to the caller.
- Routers are peers that perform generic call and event routing. Router act as Broker in publish-subscribe model.
- Broker : It act as a router and routes the messages published to a topic to all subscriber subscribed to the topic.

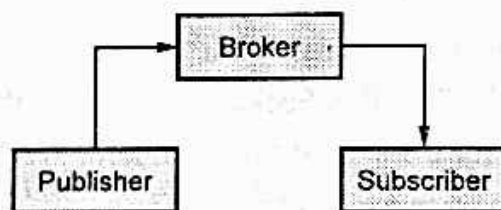


Fig. 5.3.2 (a) : Publisher-subscriber

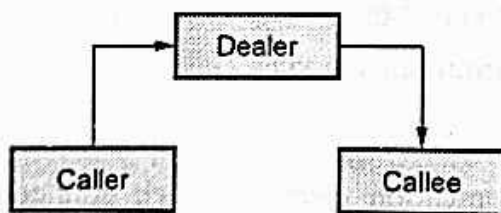


Fig. 5.3.2 (b) : RPC

- In RPC model router has the role of a broker :
 1. Dealer : It act as router and routes the RPC calls from the caller to the callee and routes results from callee to caller.
- Application code runs on client machine. Fig. 5.3.1 shows WAMP session between client and router.
- A WAMP Session connects two Peers, a Client and a Router. Each WAMP Peer MUST implement one role and MAY implement more roles. A Client MAY implement any combination of the Roles : Callee, Caller, Publisher and Subscriber.

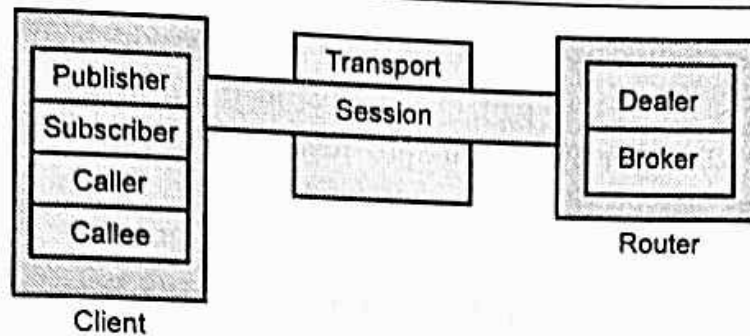


Fig. 5.3.3 : WAMP session between client and router

- Router MAY implement either or both of the Roles : Dealer and Broker.
- WAMP implementations may choose to tie the lifetime of the underlying transport connection for a WAMP connection to that of a WAMP session, i.e. establish a new transport-layer connection as part of each new session establishment. They may equally choose to allow re-use of a transport connection, allowing subsequent WAMP sessions to be established using the same transport connection.
- Fig. 5.3.4 shows the full transport connection and session lifecycle for an implementation which uses WebSocket over TCP as the transport and allows the re-use of a transport connection.

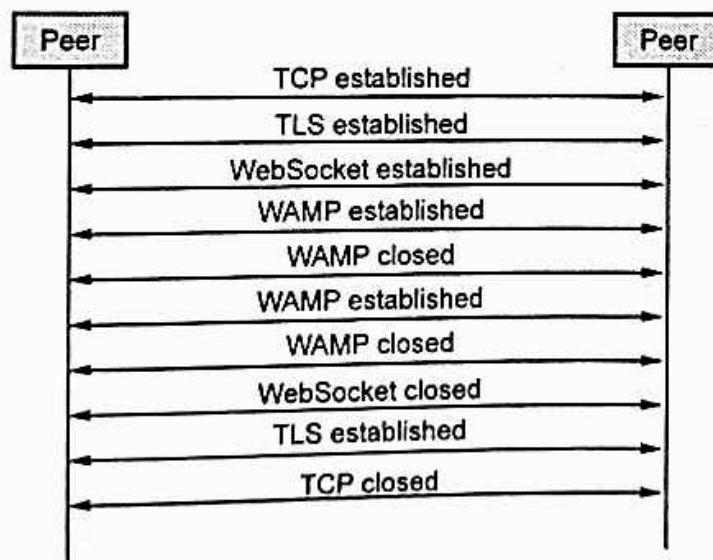


Fig. 5.3.4

❑ Autobahn

- Autobahn is an open-source real-time framework enabling implementations of the WAMP and WebSocket protocols using multiple languages that are supported by Tavendo. These languages include Autobahn Python (based on Python), Autobahn JS (based on JavaScript), Autobahn Cpp (based on C++) and Autobahn Android.

- Each language is considered a subproject with its own sub-community and developers. Autobahn Python is utilized as part of Crossbar.io itself. While these four libraries are supported by Tavendo, other libraries are provided by third-parties including ones based on other languages not supported by Tavendo such as C #, Java and PHP. Autobahn also includes a fully automated testing tool called Autobahn Testsuite.
- Here is a simple WebSocket echo server that will echo back any WebSocket message received :

```
from autobahn.twisted.websocket import WebSocketServerProtocol

class MyServerProtocol(WebSocketServerProtocol):

    def onConnect(self, request):
        print("Client connecting: {}".format(request.peer))

    def onOpen(self):
        print("WebSocket connection open.")

    def onMessage(self, payload, isBinary):
        if isBinary:
            print("Binary message received: {} bytes".format(len(payload)))
        else:
            print("Text message received: {}".format(payload.decode('utf8')))
            # echo back message verbatim
            self.sendMessage(payload, isBinary)

    def onClose(self, wasClean, code, reason):
        print("WebSocket connection closed: {}".format(reason))
```

Review Questions

1. Explain WAMP and its key concepts with diagram. **SPPU : May-18, Marks 5**
2. Draw and explain the WAMP protocol interaction between peers. **SPPU : Dec.-18, Marks 6**
3. Write the Auto Bahn Installation and setup steps. **SPPU : Dec.-18, Marks 4**
4. Explain WAMP protocol interaction between peers. **SPPU : May-19, Marks 4**
5. Draw and explain the publish-subscribe messaging using WAMP-Auto Bahn. **SPPU : Dec.-19, Marks 6**
6. With the explanation of key concepts of WAMP, draw and explain a WAMP session between client and router. **SPPU : Dec.-19, Marks 7**

5.4 Xively Cloud for IoT

➡ **SPPU : May-18, 19, Dec.-18, 19**

- Xively is a division of LogMeIn Inc. (NASDAQ : LOGM), a global, public company that provides remote access and collaboration products including Rescue, Boldchat, join.me and Cubby.
- The term IoT cloud platform is used for the cloud platforms that provide very specific services, protocol support and web-based tools for IoT applications.
- Managing a cloud IoT network is a highly complicated task. Provisioning, connecting and monitoring thousands or millions of devices is one challenge, but handling and integrating the large volumes of real-time data they produce is more difficult.
- To simplify your move to the cloud IoT space, you need a powerful but easy-to-use IoT cloud platform that lets you reduce the cost, complexity and risk of developing a connected business. Xively's cloud IoT platform makes things easy.
- With cloud IoT technology from Xively, you can :
 1. Model and develop new IoT products.
 2. Connect both new and existing products quickly and easily.
 3. Minimize the complexity and reduce the risk of running a connected business.
 4. Turn data and relationships into insight that can help you deliver greater value to customers.
- Features of Xively's cloud IoT technology :
 1. Blazing fast speed and infinitely scalable connectivity, with the hardware and software to process messages from millions of devices at sub-second speeds.
 2. Device management tools for provisioning, monitoring and updating devices.
 3. Authentication tools for managing identities and authenticating devices.
 4. Security features that offer end-to-end protection of devices and the data residing on them.
 5. Remote device support that enables connected products to autonomously communicate problems to OEMs, providing context that can be used to troubleshoot devices automatically or manually.
 6. Support for a device cloud that can be used to test new devices.
- Xively allows devices to securely connect at scale to transmit and store data.

- Following is the Python program using Raspberry Pi which will receive temperature readings every 10 seconds and display the most recent temperature reading and a graph of past readings.

```
#!/usr/bin/env python

import os
import xively
import subprocess
import time
import datetime
import requests

FEED_ID = "YOURFEEDID"
API_KEY = "YOURAPIKEY"
# initialize api client
api = xively.XivelyAPIClient(API_KEY)

# function to read the temperature from ds18b20 temperature sensor on i2cdef
read_temperature():
    tempfile = open("/sys/bus/w1/devices/YOURTEMPERATURESENSORID/w1_slave")
    thetext = tempfile.read()
    tempfile.close()
    tempdata = thetext.split("\n")[1].split(" ")[9]
    temperature = float(tempdata[2:])
    temperature = temperature / 1000
    return temperature

# function to return a datastream object. This either creates a new datastream, or
# returns an existing one
def get_datastream(feed):
    try:
```



```
datastream = feed.datastreams.get("TemperatureSensor")
return datastream

except:

    datastream = feed.datastreams.create("TemperatureSensor", tags="temperature")
    return datastream

# main program entry point - runs continuously updating our data stream with the
latest temperature reading

def run():

    feed = api.feeds.get(FEED_ID)
    datastream = get_datastream(feed)
    datastream.max_value = None
    datastream.min_value = None

    while True:

        degreesCelcius = read_temperature()
        datastream.current_value = degreesCelcius
        datastream.at = datetime.datetime.utcnow()

        try:
            datastream.update()
        except requests.HTTPError as e:
            print "HTTPError({0}): {1}".format(e.erno, e.strerror)

        time.sleep(10)

run()
```

- Steps for sending data from device to Xively :

1. Download the ArduinoIDE.
2. Install Xively and the HN P Client libraries.
3. Connect the hardware module to Arduino.
4. Login to Xively and register a new device.
5. Copy the API_KEY and the FEED_ID to the clipboard.
6. Download the example sketch for Arduino.

7. Change the SSID, the Password, the API_KEY and the FEE_ID in the source code.
 8. Compile and upload the example.
 9. Debug the program by using the Arduino Terminal.
 10. Verify the data storage in Xively.
 11. Switch on/off the light from Xively
- Xively is available for many platforms since libraries for Arduino, Android, ARM mbed, C, Electric Imp, Java, JavaScript, Objective-C, PHP, Python and Ruby are available.
 - Xively provides message bus for real time message management and routing, data services for time archiving, directory services that provides a searchable directory of objects and business services for device provisioning and management.
 - Xively device support one or more channels. Each channel enables bi-directional communication between the IoT devices and Xively cloud.

Review Questions

- | | |
|--|------------------------------------|
| 1. Write short note on : Xively cloud for IoT. | SPPU : May-18, 19, Marks 3 |
| 2. Explain python web application framework - Django. | SPPU : May-18, Marks 5 |
| 3. Explain in brief model, template and view in Django architecture. | SPPU : May-18, Marks 6 |
| 4. Briefly explain Xively Cloud for IoT. | SPPU : Dec.-18, 19, Marks 4 |

5.5 Python Web Application Framework : Django

SPPU : Dec.-18, May-19

- Django is a open source Python web development framework. Django is based on the well known Model-Template-View architecture. It provides a unified API to a database back end.
- Django is a registered trademark of the Django Software Foundation and is licensed under BSD License.

5.5.1 Django Architecture

- Django is often referred to as an MTV framework. In the MTV development pattern :
 1. M stands for "Model," the data access layer. This layer contains anything and everything about the data : How to access it, how to validate it, which behaviors it has and the relationships between the data.

2. T stands for "Template," the presentation layer. This layer contains presentation-related decisions : How something should be displayed on a Web page or other type of document.
3. V stands for "View," the business logic layer. This layer contains the logic that accesses the model and defers to the appropriate template(s). You can think of it as the bridge between models and templates.

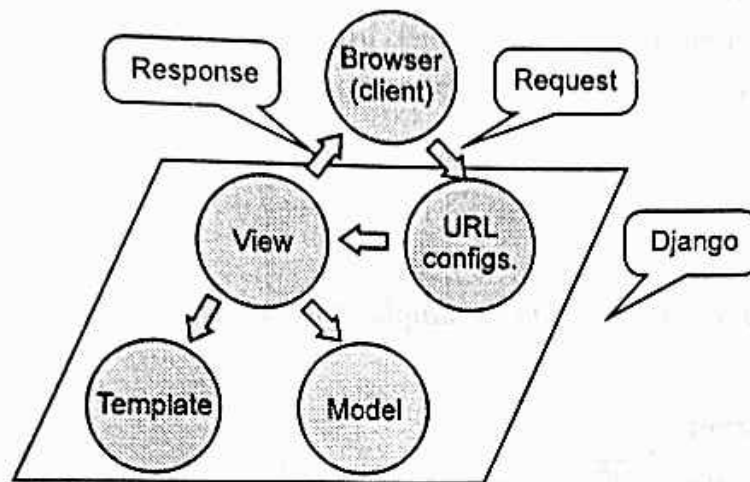


Fig. 5.5.1 : Django architecture

- Django was built using Python, an object-oriented applications development language which combines the power of systems languages, such as C/C++ and Java, with the ease and rapid development of scripting languages, such as Ruby and Visual Basic.
- DJANGO MVC - MVT Pattern : The Model-View-Template (MVT) is slightly different from MVC. In fact the main difference between the two patterns is that Django itself takes care of the controller part (Software code that controls the interactions between the model and view), leaving us with the template.

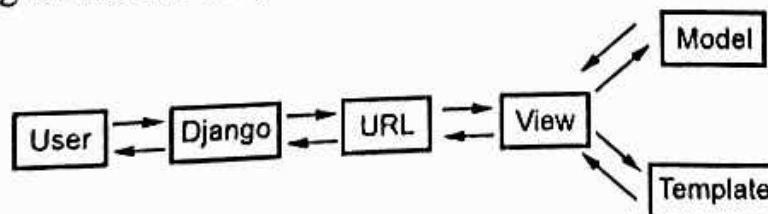


Fig. 5.5.2 : MVT/MVC

5.5.2 Starting Development with Django

- To create the project directory for your blog project, issue this django-admin.py command :
- django-admin.py startproject mysite**

- When you create projects, it creates following files :
 1. The `__init__.py` file makes this project directory a Python package, a collection of related Python modules.
 2. `manage.py` is a utility for working with this Django project. You can see from its permissions flags in the directory listing that it is executable.
 3. `settings.py` is a file containing default settings for your project. These include database information, debugging flags and other important variables. Any value in this file is available to any of your project's installed apps.
 4. `urls.py` is what's known in Django as a URLconf, a configuration file that maps URL patterns to actions your applications perform. URLconfs are an exciting and powerful feature of Django.
- Running the development server is as simple as issuing a single command. The command to run the server is
`./manage.py runserver`
- You should see something like the following
Validating models...
0 errors found.
Django version 1.0, using settings 'mysite.settings'
Development server is running at http://127.0.0.1:8000/
Quit the server with CONTROL-C
- Open that link (<http://127.0.0.1:8000/>) in your browser and you should see Django's "It Worked!" screen, as shown in Fig. 5.5.3.

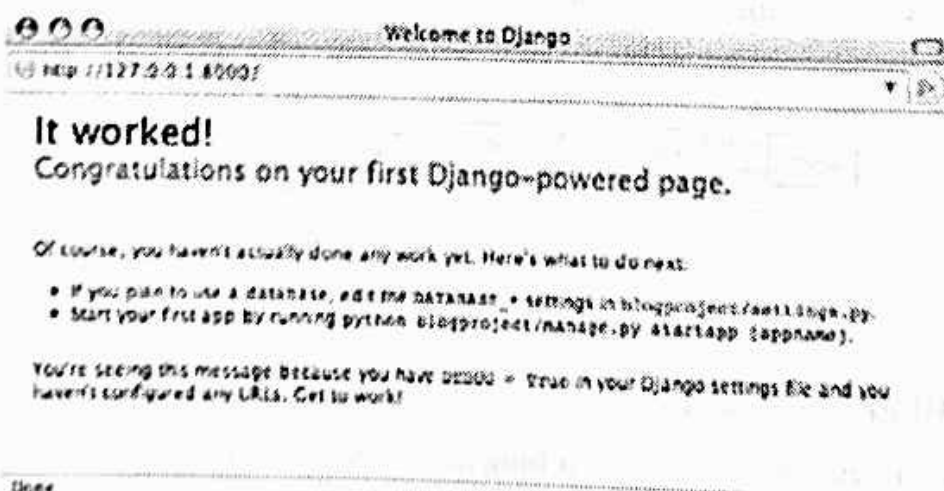


Fig. 5.5.3 : Django Initial screen

- **Steps to create new project**

- a. Create a project
- b. Start an application
- c. Create the database (MySQL, Postgresql, SQLite)
- d. Define DB settings in settings.py
- e. Define your models
- f. Add pluggable modules
- g. Write your templates
- h. Define your views
- i. Create URL mapping
- j. Test application
- k. Deploy application (Linux, Apache, mod_Python, DB)

- A project is a collection of settings for an instance of Django, including database configuration, Django-specific options and application-specific settings.

- **Setting up the database**

- If database server is not installed, then uses SQLite as the fastest and easiest way to get going. It is fast, widely available and stores its database as a single file in the file system.
- If you do have a database server like PostgreSQL, MySQL, Oracle, MSSQL and want to use it rather than SQLite, then use your database's administration tools to create a new database for your Django project.
- For using any database, then projects setting.py file is used. SQLite is also one of the most popular choices on Win32 platforms because it comes free with the Python distribution.
- Let's create a db directory as well :

`DATABASE_ENGINE = 'sqlite3'`

`DATABASE_NAME = r 'C:\py\django\db\django.db'`

How Django processes a request ?

- When an HTTP request comes in from the browser, a server-specific handler constructs the "HttpRequest" passed to later components and handles the flow of the response processing. The handler then calls any available Request or View middleware. These types of middleware are useful for augmenting incoming "HttpRequest" objects as well as providing special handling for specific types of requests. If either returns an "HttpResponse", processing bypasses the view.

- Fig. 5.5.4 shows complete flow of a Django request and response.

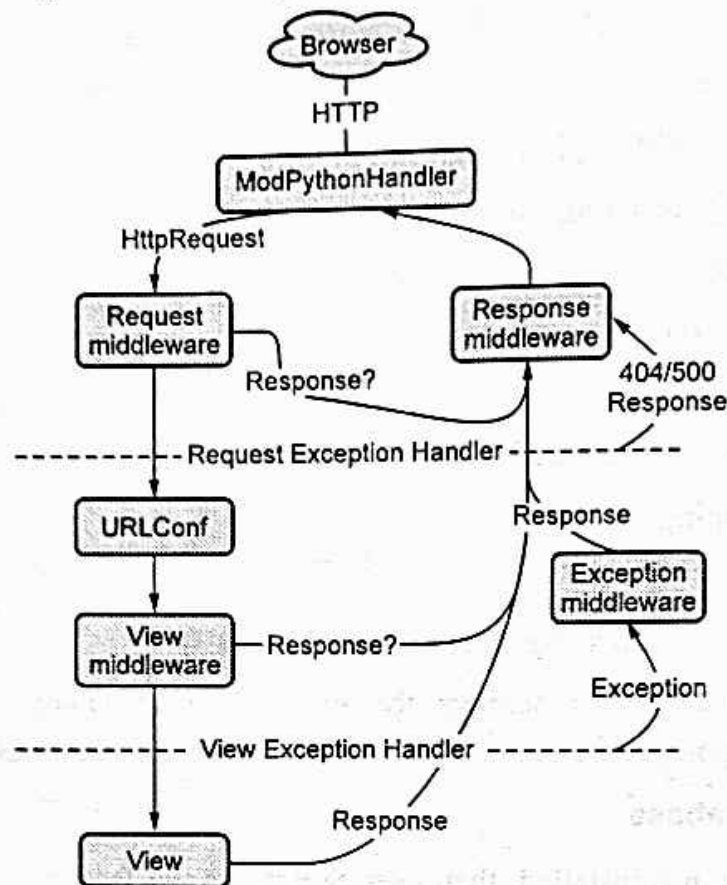


Fig. 5.5.4 : Complete flow of a Django request and response

- Bugs slip by even the best programmers, but exception middleware can help squash them. If a view function raises an exception, control passes to the exception middleware. If this middleware does not return an "HttpResponse", the exception is re-raised.
- Finally, response middleware is good for post-processing an "HttpResponse" just before it's sent to the browser or doing cleanup of request-specific resources.

☐ Template system

- A Django template is a string of text that is intended to separate the presentation of a document from its data. A template defines placeholders and various bits of basic logic (i.e., template tags) that regulate how the document should be displayed.
- To use the template system in Python code, just follow these two steps :
 - Create a template object by providing the raw template code as a string. Django also offers a way to create template objects by designating the path to a template file on the filesystem; we will examine that in a bit.
 - Call the render() method of the template object with a given set of variables. This returns a fully rendered template as a string, with all of the variables and block tags evaluated according to the context.

- The easiest way to create a template object is to instantiate it directly. The template class lives in the "django.template" module and the constructor takes one argument, the raw template code.

Review Questions

1. Explain the python web application framework - Django.

SPPU : Dec.-18, Marks 6

2. What is Django ? Explain model, template and view in Django.

SPPU : May-19, Marks 6

5.6 Amazon Web Services for IoT



SPPU : May-18, 19, Dec.-18, 19

- American international multibillion dollar electronic commerce company with headquarters in Seattle, Washington, USA. Started in 1995 by Jeff Bezos as an online bookstore but soon diversified, selling DVDs, VHSs, CDs, video and MP3 downloads/streaming, software, video games, electronics, apparel, furniture, food, toys and jewelry.
- The company also produces consumer electronics : Kindle e-book reader and the Kindle Fire tablet computer. In 2006, Amazon officially launched the Amazon Web Services (AWS) to become a major provider of cloud computing services.
- Amazon Web Services (AWS) is a secure cloud services platform, offering compute power, database storage, content delivery and other functionality to help businesses scale and grow.
- In 2006, Amazon Web Services (AWS) began offering IT infrastructure services to businesses in the form of web services, now commonly known as cloud computing.
- Today, AWS provides a highly reliable, scalable, low-cost infrastructure platform in the cloud that powers hundreds of thousands of businesses in 190 countries around the world.

What is Amazon Web Services ?

- Amazon Web Services (AWS) is a collection of remote computing services (web services) that together make up a cloud computing platform, offered over the Internet by Amazon.com.
- The AWS Cloud infrastructure is built around Regions and Availability Zones (AZs). A Region is a physical location in the world where we have multiple AZs. AZs consist of one or more discrete data centers, each with redundant power, networking and connectivity, housed in separate facilities.
- These AZs offer you the ability to operate production applications and databases that are more highly available, fault tolerant and scalable than would be possible from a single data center.

- The AWS cloud operates 42 AZs within 16 geographic regions around the world, with five more availability zones and two more regions coming online in 2017.
- Each availability zone is designed as an independent failure zone. This means that availability zones are physically separated within a typical metropolitan region and are located in lower risk flood plains.



5.6.1 Amazon EC2

- Amazon Elastic Compute Cloud (Amazon EC2) is a web service that provides resizable compute capacity in the cloud. It is designed to make web-scale computing easier for developers and system administrators.
- The Amazon EC2 simple web service interface allows you to obtain and configure capacity with minimal friction. It provides you with complete control of your computing resources and lets you run on Amazon's proven computing environment.
- Amazon EC2 reduces the time required to obtain and boot new server instances (called Amazon EC2 instances) to minutes, allowing you to quickly scale capacity, both up and down, as your computing requirements change.
- Amazon EC2 changes the economics of computing by allowing you to pay only for capacity that you actually use. Amazon EC2 provides developers and system administrators the tools to build failure resilient applications and isolate themselves from common failure scenarios.
- EC2 allows creating Virtual Machines (VM) on-demand. Pre-configured template Amazon Machine Image (AMI) can be used get running immediately. Creating and sharing your own AMI is also possible via the AWS Marketplace.
- Amazon Machine Image (AMI) is a template for software configuration (Operating System, Application Server, and Applications). Fig. 5.6.1 shows AMI and instance.

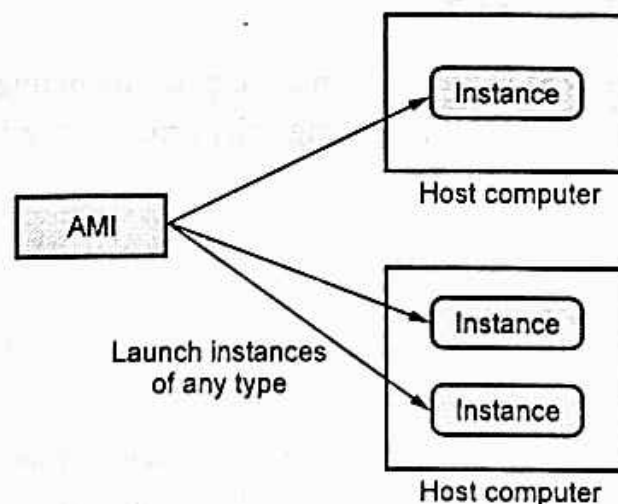


Fig. 5.6.1 : AMI and instance

- Instance is a AMI running on virtual servers in the cloud. Each instance type offers different compute and memory facilities. Create an Amazon Machine Image (AMI) containing your applications, libraries, data and associated configuration settings. Or use pre-configured, templated images to get up and running immediately.
- Auto scaling allows automatically scale of the capacity up seamlessly during demand spikes to maintain performance and scales down during demand lulls to minimize costs.
- Elastic load balancing automatically distributes incoming application traffic across multiple Amazon EC2 instances. It provide tools to build failure resilient applications by launching application instances in separate availability zones.
- Pay only for resources actually consume, instance-hours. VM Import/Export enables you to easily import virtual machine images from your existing environment to Amazon EC2 instances and export them back at any time.
- Boto is a Python package that provides programmatic connectivity to Amazon Web Services.

❑ Launching an EC2 instance :

```
#!/usr/bin/python
import boto.ec2
conn = boto.ec2.connect_to_region("us-west-2")
conn.run_instances(
    'ami-6ac2a85a',
    key_name='nitheesh_oregon',
    instance_type='t1.micro',
    security_groups=['nitheesh_oregon']
)
```

❑ Stop instances :

```
#!/usr/bin/python
import boto.ec2
conn = boto.ec2.connect_to_region("us-west-2")
conn.stop_instances(instance_ids=['instance-id-1',
'instance-id-2'])
```

- Boto supports more than fifty Amazon services, running the whole range from compute, database, application and payments and billing.
- EC2 functions :
 1. Load variety of operating system.
 2. Install custom applications.
 3. Manage network access permission.
 4. Run image using as many/few systems as you desire.
- EC2 advantages :
 1. Amazon EC2 enables you to increase or decrease capacity within minutes.
 2. User have complete control of your Amazon EC2 instances.
 3. Support flexible cloud hosting services
 4. Secure : Amazon EC2 works in conjunction with Amazon VPC to provide security and robust networking functionality.
 5. Reliable : Amazon EC2 offers a highly reliable environment where replacement instances can be rapidly and predictably commissioned.



5.6.2 Amazon AutoScaling

- Auto scaling is a service offered by Amazon Web Services that helps customers automatically adjust their Amazon EC2 capacity according to the current load. A typical use case consists of a web application served by multiple web servers that are deployed across multiple availability zones. An Elastic Load Balancer (ELB) would distribute network traffic across this group of web servers.
- AutoScaling can be used for auto scaling IoT applications and IoT platforms deployed on Amazon EC2. Auto scaling allows user to specify a server group, called an Auto Scaling Group (ASG), in which you define :
 1. A minimum number of servers to run.
 2. A maximum number of servers to run.
 3. Optionally, an initial number of servers to run.
 4. The AZs in which you want the servers to run.

- Fig. 5.6.2 shows concept of AutoScaling.

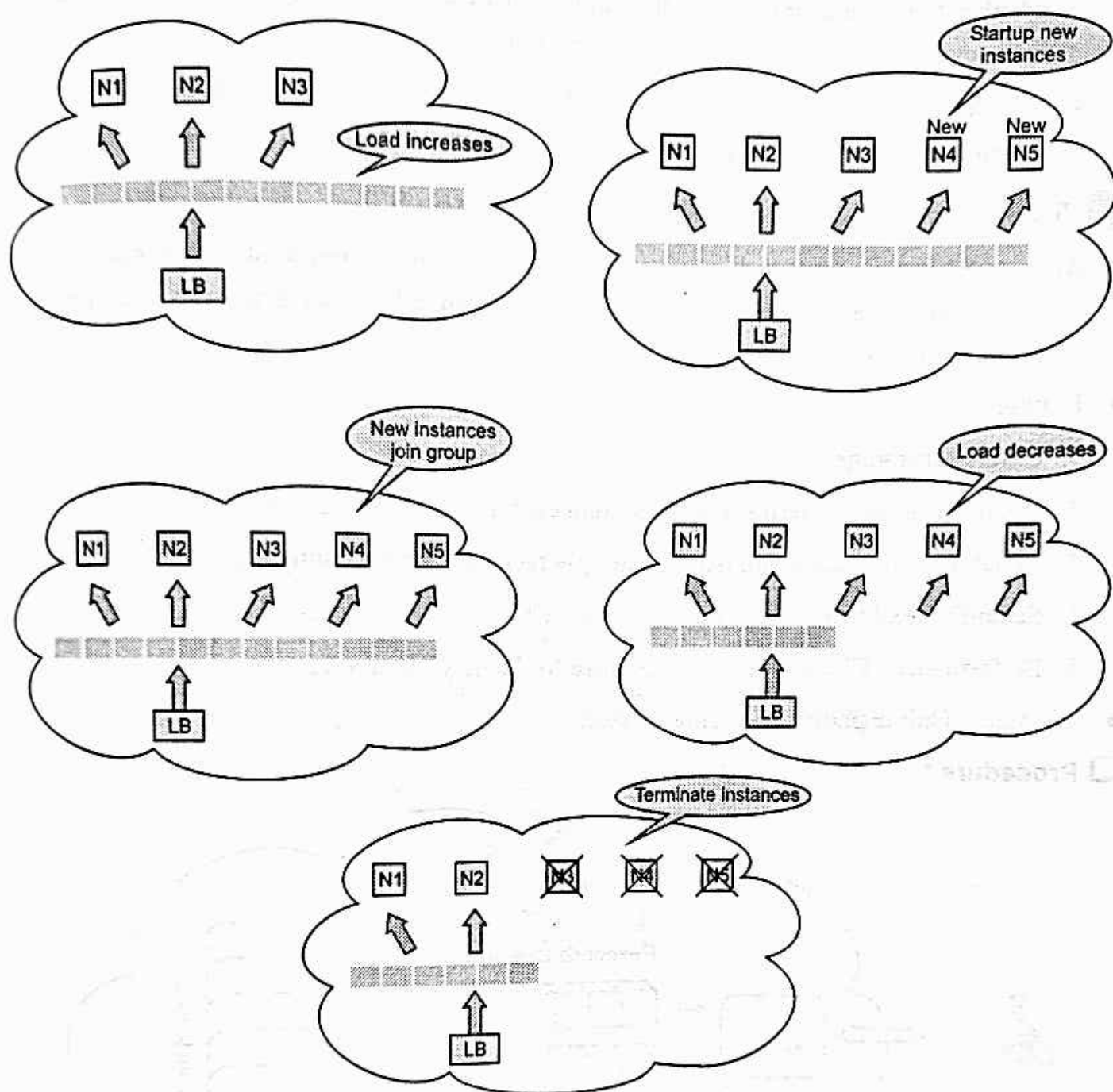


Fig. 5.6.2 : Auto scaling

- Auto scaling consists of the following four components :
 1. Launch Configuration (LC) : LC tells auto scaling how to configure the EC2 instances it launches.
 2. Auto Scaling Group (ASG) : It defines where the EC2 instances described in a LC will run, what the minimum number of instances must be and what the maximum number of instances must be.

3. Auto Scaling Policy (ASP) : Defines how an ASG should be changed, which includes whether to increase or decrease the number of EC2 instances by a certain percentage or by an exact number or to change to an exact number of EC2 instances.
4. Scheduled Action (SA) : It is an action that is triggered at a specific time to decrease or increase the number of EC2 instances running in an ASG.

5.6.3 Amazon Simple Storage Service (Amazon S3)

- Amazon S3 has a simple web services interface that you can use to store and retrieve any amount of data, at any time, from anywhere on the web. S3 can serve as a raw data store for IoT systems for storing raw data, such as sensor data, log data, audio and video data.
- Features :
 1. Unlimited storage
 2. Highly scalable : In terms of storage, request rate and concurrent users.
 3. Reliable : Store redundant data in multiple facilities and on multiple devices.
 4. Secure : Flexibility to control who / how / when / where to access the data.
 5. Performance : Choose region to optimize for latency / minimize costs.
- Example : Online photo processing service.

Procedure :

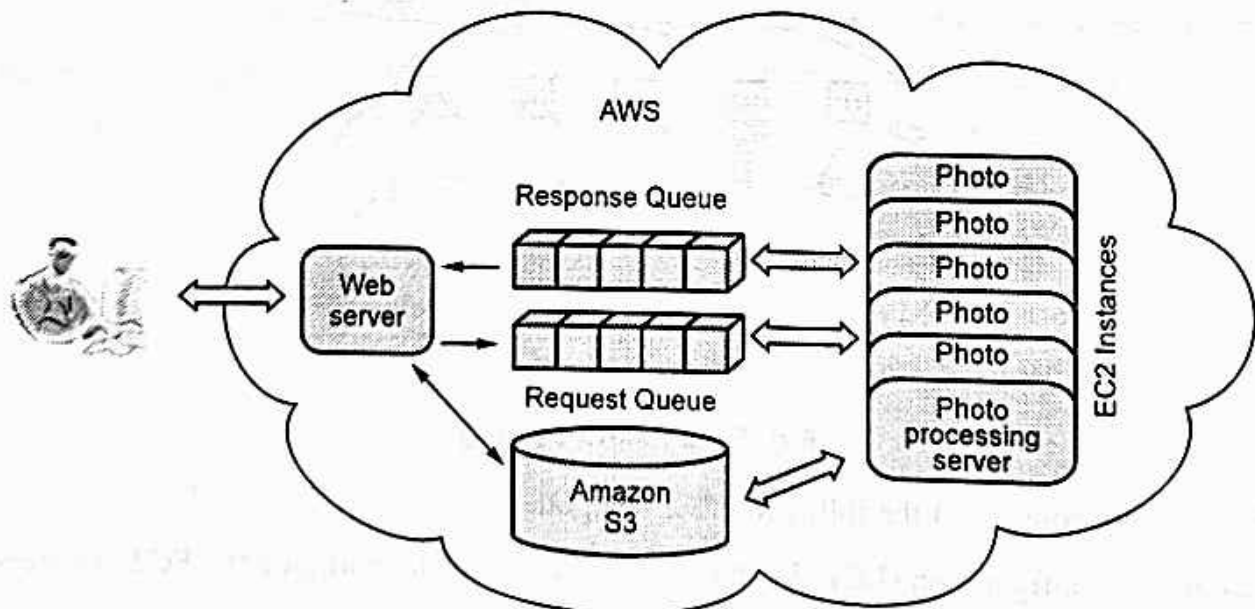


Fig. 5.6.3 : Amazon S3 working

1. Web server receive request.
 2. Put request message in the queue.
 3. Pictures stored in S3.
 4. Multiple EC2 instances run photo processing.
 5. Put back in the queue.
 6. Return
- Store data on Amazon's distributed system containing multiple servers within Amazon's data center locations. Amazon doesn't offer you a GUI based tool to access your data. You can use one of the several tools online or build one through APIs.
 - Amazon EC2 provides three type of storage option : Amazon EBS, Amazon S3 and Instance Storage. Amazon EBS (Elastic Block Store) provides with persistent, block-level storage. Basically additional hard disk that you can attach to instance. It suitable for apps which require database, file system, block level storage.

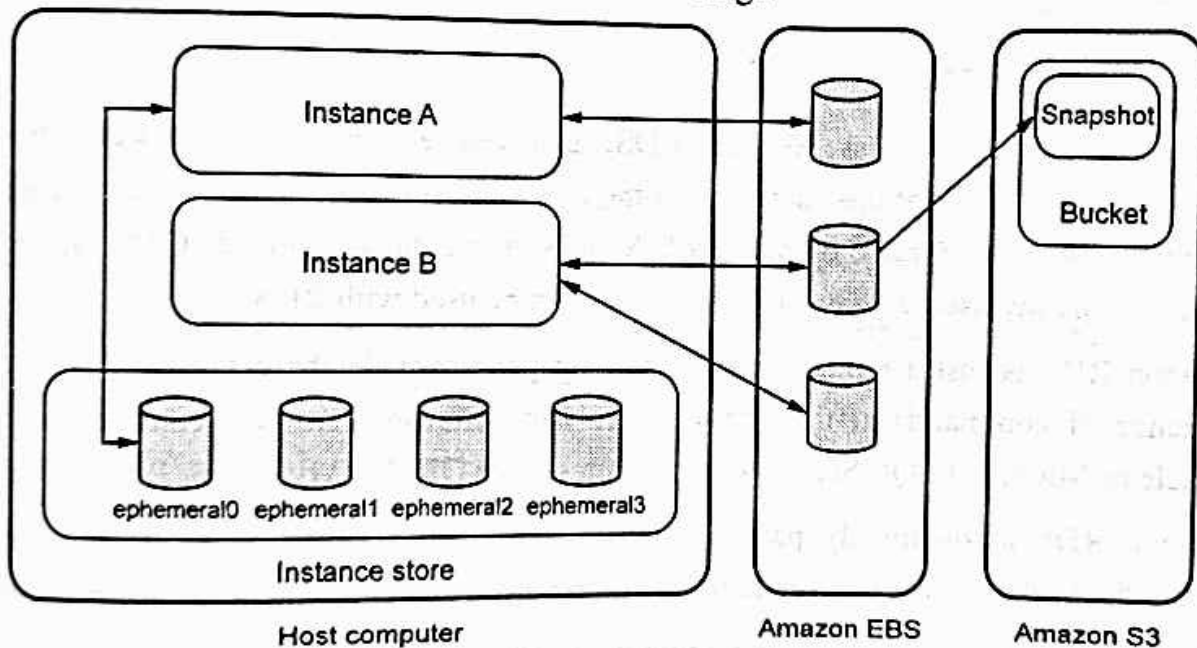


Fig. 5.6.4 : Amazon EBS and S3

- A **bucket** is a container for objects stored in Amazon S3. Every object is contained in a bucket. For example, if the object named "photos/puppy.jpg" is stored in the rakshita bucket, then it is addressable using the URL
<http://rakshita.s3.amazonaws.com/photos/puppy.jpg>
- Buckets serve several purposes : They organize the Amazon S3 namespace at the highest level, they identify the account responsible for storage and data transfer charges, they play a role in access control and they serve as the unit of aggregation for usage reporting.

- **Objects** are the fundamental entities stored in Amazon S3. Objects consist of object data and metadata. The data portion is opaque to Amazon S3. The metadata is a set of name-value pairs that describe the object. These include some default metadata, such as the date last modified and standard HTTP metadata, such as content-type. You can also specify custom metadata at the time the object is stored.
- A **key** is the unique identifier for an object within a bucket. Every object in a bucket has exactly one key. Because the combination of a bucket, key and version ID uniquely identify each object, Amazon S3 can be thought of as a basic data map between "bucket + key + version" and the object itself. Every object in Amazon S3 can be uniquely addressed through the combination of the web service endpoint, bucket name, key and optionally, a version.
- **Regions** : You can choose the geographical region where Amazon S3 will store the buckets you create. Objects stored in a region never leave the region unless you explicitly transfer them to another region.

5.6.4 Amazon RDS

- Amazon Relational Database Service (RDS) is a web service that makes it easy to set up, operate and scale a relational database in the cloud. RDS gives access to the capabilities of a familiar MySQL, Oracle or Microsoft SQL Server database engine. Code, applications and tools already used with existing databases can be used with RDS.
- Amazon RDS is just a replacement to running your own database server. With a simple sequence of commands at the console, you can choose from two commercial DBMS (Oracle or Microsoft SQL Server) or two open source DBMS (MySQL or PostgreSQL).
- Amazon RDS automatically patches the database software and backs up the database, storing the backups for a user-defined retention period and enabling point-in-time recovery.
- Amazon RDS provides scaling the compute resources or storage capacity associated with the Database Instance. Pay only for the resources actually consumed, based on the DB Instance hours consumed, database storage, backup storage and data transfer.
- On-Demand DB Instances let you pay for compute capacity by the hour with no long-term commitments. Reserved DB Instances give the option to make a low, one-time payment for each DB Instance and in turn receive a significant discount on the hourly usage charge for that DB Instance.
- Amazon RDS provides you six familiar database engines to choose from, including Amazon Aurora, PostgreSQL, MySQL, MariaDB, Oracle and Microsoft SQL Server.

- Create the actual RDS instance :

```
instance = conn.create_dbinstance ( id=identifier, instance_class="db." +  
instance_class, allocated_storage=allocated_storage,  
engine=engine, db_name=db_name, master_username=master_username,  
master_password=master_password,  
db_subnet_group_name=identifier, vpc_security_groups=security_groups,  
port=port)
```

□ Advantages :

1. Fast and easy to administer : Amazon RDS makes it easy to go from project conception to deployment.
2. Available and durable : Amazon RDS runs on the same highly reliable infrastructure used by other Amazon Web Services.
3. Highly scalable : You can scale your database's compute and storage resources.
4. Secure : Amazon RDS makes it easy to control network access to your database.

5.6.5 Amazon DynamoDB

- **DynamoDB** is a fast, fully managed **NoSQL database service** that makes it simple and cost-effective to store and retrieve any amount of data and serve any level of request traffic.
- All data items are stored on **Solid State Drives (SSDs)** and are **replicated** across **3 Availability Zones** for high availability and durability.
- **DynamoDB tables do not have fixed schemas** and each item may have a **different number of attributes**.
- **DynamoDB has no upfront costs** and implements a **pay as you go** plan as a. a **flat hourly rate** based on the **capacity reserved**.
- With **DynamoDB**, you can create database tables that can store and retrieve any amount of data and serve any level of request traffic. You can scale up or scale down your tables' throughput capacity without downtime or performance degradation and use the **AWS management console** to monitor resource utilization and performance metrics.
- **DynamoDB** allows you to delete expired items from tables automatically to help you reduce storage usage and the cost of storing data that is no longer relevant.

- The first step in accessing DynamoDB is to create a connection to the service :

```
import boto.dynamodb

conn = boto.dynamodb.connect_to_region('us-west-2',
aws_access_key_id='<YOUR_AWS_KEY_ID>',
aws_secret_access_key='<YOUR_AWS_SECRET_KEY>')
```

- Python program for creating a DynamoDB table with other operation :

```
import boto.dynamodb
from boto.dynamodb.condition import *
connection = boto.dynamodb.connect_to_region('eu-west-1')
table = connection.get_table('table')

id = '1'
timestamp = 1234
attrs = {
    'key1': 'value1',
    'key2': set(['value2', 'value3'])
}

# create
item = table.new_item(hash_key=id, range_key=timestamp, attrs=attrs)
item.put()

# read
item = table.get_item(hash_key=id)
key2 = list(item['key2'])

# update
item['key1'] = 'foo'
item['key3'] = 'bar'
```



```
item.put()

# query
table.query(hash_key=id, range_key_condition=LT(1500))

# scan
table.scan(scan_filter={'key1': EQ('foo')})

# delete
item = table.get_item(hash_key=id)
item.delete()
```

❑ Advantages :

1. Flexible : Amazon DynamoDB supports both document and key-value data structures.
2. Fully managed : Amazon DynamoDB is a fully managed cloud NoSQL database service.
3. Highly scalable : When you create a table, simply specify how much request capacity you require.
4. Event-driven programming : Amazon DynamoDB integrates with AWS lambda to provide triggers

📖 5.6.6 Amazon Kinesis

- Amazon Kinesis is a platform for streaming data on AWS, making it easy to load and analyze streaming data and also providing the ability for you to build custom streaming data applications for specialized needs.
- Amazon Kinesis services make it easy to work with real-time streaming data in the AWS cloud.
- Kinesis scales automatically to handle high volume streaming data coming from large number of sources. Kinesis allows rapid and continuous data intake and support data blobs of size upto 50 Kb.

- Typical Amazon Kinesis streams application reads data from an Amazon Kinesis stream as data records. These applications can use the Amazon Kinesis client library and they can run on Amazon EC2 instances.
- The processed records can be sent to dashboards, used to generate alerts, dynamically change pricing and advertising strategies or send data to a variety of other AWS services.



5.6.7 Amazon SQS

- Amazon Simple Queue Service (SQS) is used in distributed IoT applications. SQS is intended to provide a highly scalable hosted message queue that resolves issues arising from the common producer-consumer problem or connectivity between producer and consumer.
- Amazon SQS offers standard as the default queue type. Standard queues can support a nearly unlimited number of Transactions Per Second (TPS) per API action. Standard queues support at-least-once message delivery.
- Amazon SQS stores copies of your messages on multiple servers for redundancy and high availability. On rare occasions one of the servers that stores a copy of a message might be unavailable when you receive or delete a message.
- SQS does not automatically delete messages once they are sent. When a message is delivered, a receipt handle is generated for that delivery and sent to the recipient. These receipts are not sent with the message but in addition to it. SQS requires the recipient to provide the receipt in order to delete a message. This feature is new as of 2008 where only the message ID was required for message deletion. Because the system is distributed, a message may be sent more than once. In this case, the most recent receipt handle is needed to delete the message.
- Once a message is delivered, it has a visibility timeout to prevent other components from consuming it. The "clock" for the visibility timeout starts once a message is sent, the default time being 30 seconds. If the queue is not told to delete the message during this time, the message becomes visible again and will be resent.



5.6.8 Amazon Elastic MapReduce (EMR)

- Amazon EMR provides a managed Hadoop framework that makes it easy, fast and cost-effective to process vast amounts of data across dynamically scalable EC2 instances.
- MapReduce : A programming model to facilitate the development and execution of distributed tasks.

- The programmer defines the program logic as two functions :
 - a. Map transforms the input into key-value pairs to process.
 - b. Reduce aggregates the list of values for each key.
- The MapReduce environment takes in charge distribution aspects. A complex program can be decomposed as a succession of Map and Reduce tasks. Higher-level languages (Pig, Hive, etc.) help with writing distributed applications.
- MapReduce is a parallel programming model especially dedicated for complex and distributed computations, which has been derived from the functional paradigm.
- In general, MapReduce processing is composed of two consecutive stages, which for most problems are repeated iteratively : The map and the reduce phase.
- Amazon EMR securely and reliably handles a broad set of big data use cases, including log analysis, web indexing, data transformations (ETL), machine learning, financial analysis, scientific simulation and bioinformatics.
- Map processes the data on hosts in parallel, whereas the reduce aggregates the results. At each iteration independently, the whole data is split into chunks, which, in turn, are used as the input for mappers.
- Each chunk may be processed by only one mapper. Once the data is processed by mappers, they can emit View the MathML source(key,value) pairs to the reduce phase.
- Before the reduce phase the pairs are sorted and collected according to the View the MathML sourcekey values, therefore each reducer gets the list of values related to a given View the MathML sourcekey. The consolidated output of the reduce phase is saved into the distributed file system.

□ Programming model

- The computation takes a set of input key/value pairs and produces a set of output key/value pairs. The user of the MapReduce library expresses the computation as two functions : Map and Reduce.
- Map, written by the user, takes an input pair and produces a set of intermediate key/value pairs. The MapReduce library groups together all intermediate values associated with the same intermediate key "I" and passes them to the reduce function.
- The reduce function, also written by the user, accepts an intermediate key I and a set of values for that key. It merges together these values to form a possibly smaller set of values.

- Typically just zero or one output value is produced per reduce invocation. The intermediate values are supplied to the user's reduce function via an iterator. This allows us to handle lists of values that are too large to fit in memory.
- Fig. 5.6.5 shows the overall flow of a MapReduce operation. When the user program calls the MapReduce function, the following sequence of actions occurs :

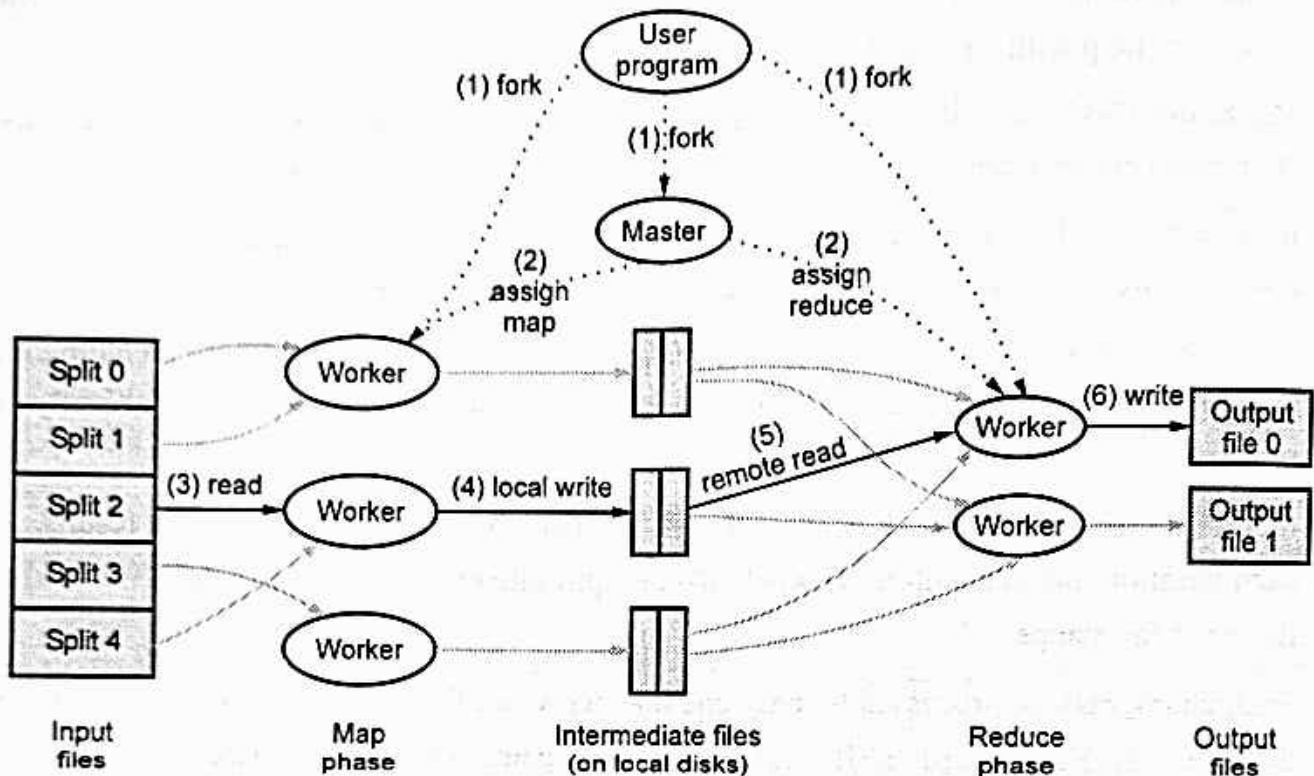


Fig. 5.6.5 : MapReduce execution

1. MapReduce library in the user program first splits the input files into M pieces of typically 16 MB to 64 MB per piece. It then starts up many copies of the program on a cluster of machines.
2. One of the copies of the program is special i.e. master. The rest are workers that are assigned work by the master. There are M map tasks and R reduce tasks to assign. The master picks idle workers and assigns each one a map task or a reduce task.
3. A worker who is assigned a map task reads the contents of the corresponding input split. It parses key/value pairs out of the input data and passes each pair to the user-defined Map function. The intermediate key/value pairs produced by the Map function are buffered in memory.
4. Periodically, the buffered pairs are written to local disk, partitioned into R regions by the partitioning function. The locations of these buffered pairs on the local disk are passed back to the master, who is responsible for forwarding these locations to the reduce workers.

5. When a reduce worker is notified by the master about these locations, it uses remote procedure calls to read the buffered data from the local disks of the map workers. When a reduce worker has read all intermediate data, it sorts it by the intermediate keys so that all occurrences of the same key are grouped together. The sorting is needed because typically many different keys map to the same reduce task. If the amount of intermediate data is too large to fit in memory, an external sort is used.
6. The reduce worker iterates over the sorted intermediate data and for each unique intermediate key encountered, it passes the key and the corresponding set of intermediate values to the user's Reduce function. The output of the Reduce function is appended to a final output file for this reduce partition.
7. When all map tasks and reduce tasks have been completed, the master wakes up the user program. At this point, the MapReduce call in the user program returns back to the user code.

Review Questions

1. Write short note on : Amazon Auto Scaling.

SPPU : May-18, 19, Marks 3

2. List and explain various Amazon / Web Services for IoT.

SPPU : Dec.-18, Marks 7

3. Write short note on : Amazon EMR.

SPPU : May-19, Marks 3

5.7 Sky Net IoT Messaging Platform

- SkyNet is an open source instant messaging service platform for IoT. By using the SkyNet platform, we are able to save a ton of work by not having to develop our own communication tools, authentication scheme or backend service to handle the streams of data. Instead, our devices simply turn on, authenticate and connect with SkyNet automatically and we are able to start passing data securely.
- Skynet is connected to cloud database and the IoT devices through communication protocols. This platform has the ability to register network devices, to store, update and exchange information.
- SkyNet API support both HTTP REST and real time WebSocket. It allows you to register devices on the network. Device includes any type of sensor, cloud resources, smart home devices and drones. When nodes and devices register with Skynet, they are assigned a unique id known as a UUID along with a security token.
- Upon connecting your node or device to Skynet, you can query and update devices on the network and send machine-to-machine (M2M) messages in an RPC-style fashion.



5.8 RESTful Web Service

**SPPU : March-20**

- A RESTful API is an architectural style for an Application Program Interface (API) that uses HTTP requests to access and use data. That data can be used to GET, PUT, POST and DELETE data types, which refers to the reading, updating, creating and deleting of operations concerning resources.
- A large part of the interoperability, scale, and control for IoT can be achieved through API management. Standards-based design patterns for Web APIs, API management, and a RESTful architecture provide tremendous value in simplifying the task of interoperability across heterogeneous systems handling vast amounts of data.
- Representational State Transfer (REST) APIs follow the request-response communication model.
- REST is a set of architectural constraints, not a protocol or a standard. API developers can implement REST in a variety of ways.
- REST requires that a client make a request to the server in order to retrieve or modify data on the server. A request generally consists of :
 - a) An HTTP verb, which defines what kind of operation to perform
 - b) A header, which allows the client to pass along information about the request
 - c) A path to a resource
 - d) An optional message body containing data
- Important REST principles :
 - a) Stateless : No client context stored on the server, each request is complete.
 - b) Cacheable : Responses explicitly indicate their cacheability.
 - c) Layered System : Client cannot tell if connected directly to the server (e.g. reverse proxies).
 - d) URIs : Resources are identified using Uniform Resource Identifiers (URIs)
- When a client request is made via a RESTful API, it transfers a representation of the state of the resource to the requester or endpoint. This information, or representation, is delivered in one of several formats via HTTP : JSON, HTML, XLT, Python, PHP, or plain text.
- JSON is the most generally popular programming language to use because, despite its name, it's language-agnostic, as well as readable by both humans and machines.

- Something else to keep in mind : Headers and parameters are also important in the HTTP methods of a RESTful API HTTP request, as they contain important identifier information as to the request's metadata, authorization, Uniform Resource Identifier (URI), caching, cookies, and more. There are request headers and response headers, each with their own HTTP connection information and status codes.
- Applications conforming to the REST constraints can be called RESTful. RESTful systems typically communicate over HTTP with the same methods (GET, POST, PUT, DELETE etc) that browsers use to retrieve web pages and to send data to remote servers.
 1. **Client-Server** : requires that a service offer one or more operations and that services wait for clients to request these operations.
 2. **Stateless** : requires communication between service consumer (client) and service provider (server) to be stateless.
 3. **Cache** : requires responses to be clearly labeled as cacheable or non-cacheable.
 4. **Interface** : requires all service providers and consumers within a REST-compliant architecture to share a single common interface for all operations.
 5. **Layered System** : requires the ability to add or remove intermediaries at runtime without disrupting the system.
 6. **Code-on-Demand** : allows logic within clients (such as Web browsers) to be updated independently from server-side logic using executable code shipped from service providers to consumers.
- Each client request and server response is a message, and REST-compliant applications expect each message to be self-descriptive. That means each message must contain all the information necessary to complete the task. Other ways to describe this kind of message are "stateless" or "context-free." Each message passed between client and server can have a body and metadata.

Review Question

1. Compare the REST and Web Socket protocol.

SPPU : March - 20, Marks 5

 5.9 gRPC

- gRPC is a recursive abbreviation that expands to gRPC Remote Procedure Call.
- gRPC is a high performance, open-source RPC framework initially developed by Google. The framework is based on a client-server model of remote procedure calls. A client application can directly call methods on a server application as if it was a local object.

- gRPC is designed to make the clients believe that the server is on the same machine. Clients invoke a method on the Stub, which gets transparently handled by the underlying protocol.
- Fig. 5.9.1 shows gRPC.

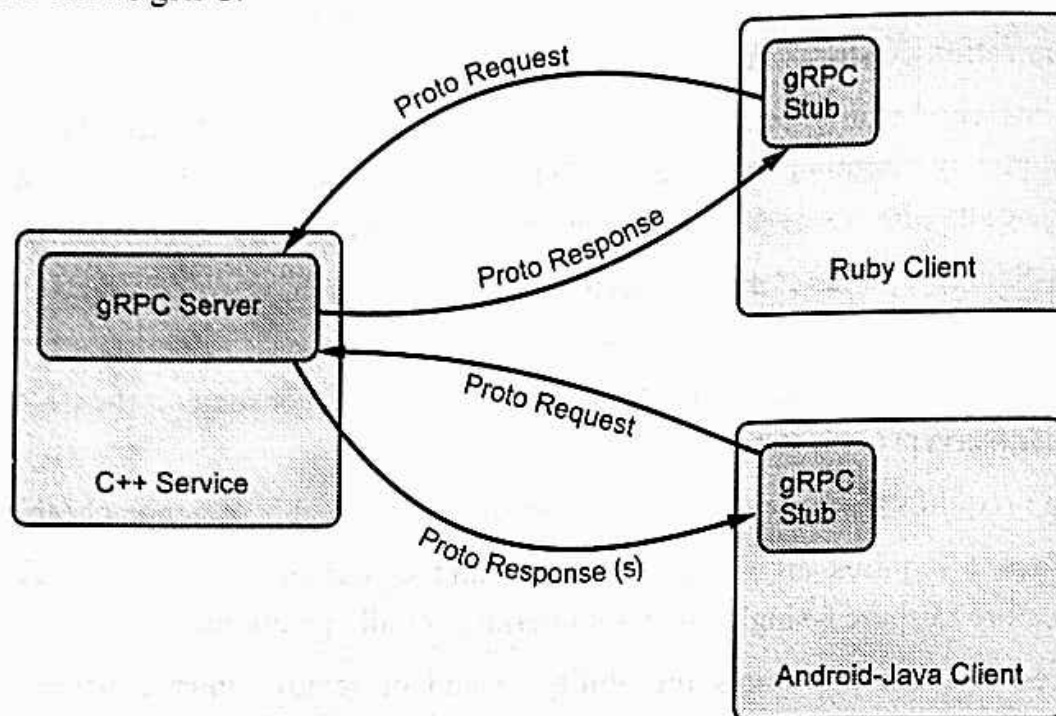


Fig. 5.9.1 : gRPC

- gRPC is a technology for implementing RPC APIs that uses HTTP 2.0 as its underlying transport protocol.
- The main advantage of developing services and clients with gRPC is that your service code or client side code doesn't need to worry about parsing JSON or similar text-based message formats.

5.10 Simple Object Access Protocol

- SOAP is the standard messaging protocol used by web services. SOAP codifies the use of XML as an encoding scheme for request and response parameters using HTTP as a means for transport.
- SOAP is a lightweight protocol intended for exchanging structured information in a decentralized, distributed environment.
- SOAP can be used over any transport protocol such as TCP, HTTP, and SMTP.
- SOAP is based on message exchanges. Messages are seen as envelopes where the application encloses the data to be sent.

- A SOAP message consists of an <Envelope> element containing an optional <Header> and a mandatory <Body> element. The contents of these elements are application defined and not a part of the SOAP specification.
- SOAP <Header> contains blocks of information relevant to how the message is to be processed. This helps pass information in SOAP messages that is not for the application but for the SOAP engine
- The SOAP <Body> is where the main end-to-end information conveyed in a SOAP message must be carried.
- A SOAP message travels along the message path from a sender to a receiver. All SOAP messages start with an initial sender, which creates the SOAP message, and end with an ultimate receiver. SOAP message consists of three parts: SOAP Envelope, SOAP Header (optional) and SOAP Body.
- The SOAP Envelope construct defines an overall framework for expressing what is in a message and who should deal with it. Fig. 5.10.1 shows SOAP messages.

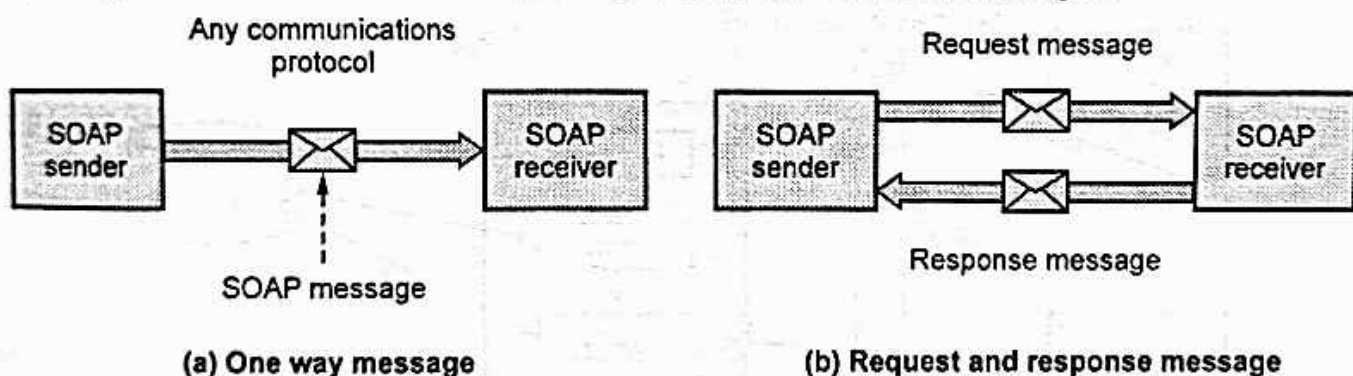


Fig. 5.10.1 : SOAP Message

- SOAP supports two possible communication styles :
 - a) Remote procedure call (RPC) and
 - b) Document (or message).
- A remote procedure call (RPC)-style Web service appears as a remote object to a client application. The interaction between a client and an RPC-style Web service centers around a service-specific interface. Clients express their request as a method call with a set of arguments, which returns a response containing a return value. Fig. 5.10.2 shows RPC communication styles.

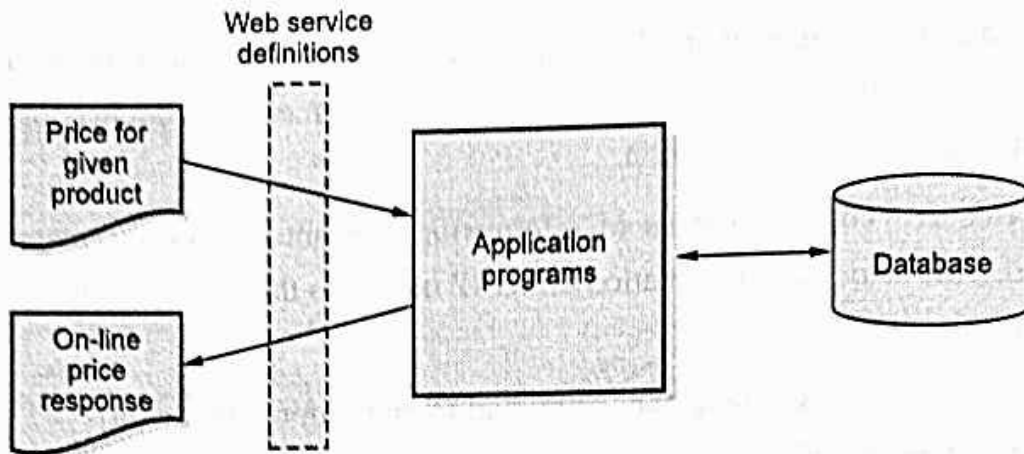


Fig. 5.10.2 : RPC styles

- In the document-style of messaging, the SOAP <Body> contains an XML document fragment. The <Body> element reflects no explicit XML structure. The SOAP run-time environment accepts the SOAP <Body> element as it stands and hands it over to the application it is destined for unchanged. There may or may not be a response associated with this message. Fig. 5.10.3 shows document style of messaging.

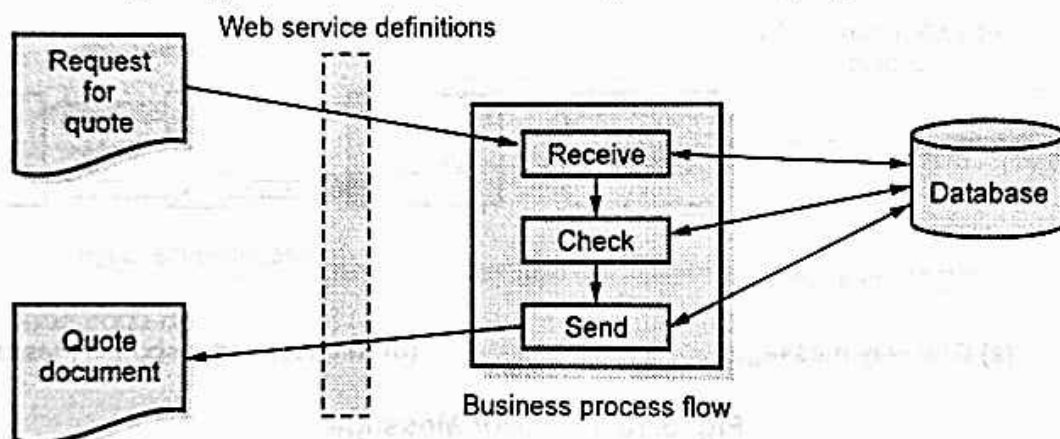


Fig. 5.10.3 : Document style of messaging

□ SOAP and HTTP

- The typical binding for SOAP is HTTP. SOAP can use GET or POST.
- With GET, the request is not a SOAP message but the response is a SOAP message, with POST both request and response are SOAP messages.
- SOAP uses the same error and status codes as those used in HTTP so that HTTP responses can be directly interpreted by a SOAP module.

☐ Advantages of SOAP

- a) Simplicity
- b) Portability
- c) Firewall friendliness
- d) Use of open standards
- e) Interoperability
- f) Universal acceptance.

☐ Disadvantages of SOAP

- a) Too much reliance on HTTP
- b) Statelessness
- c) Serialization by value and not by reference